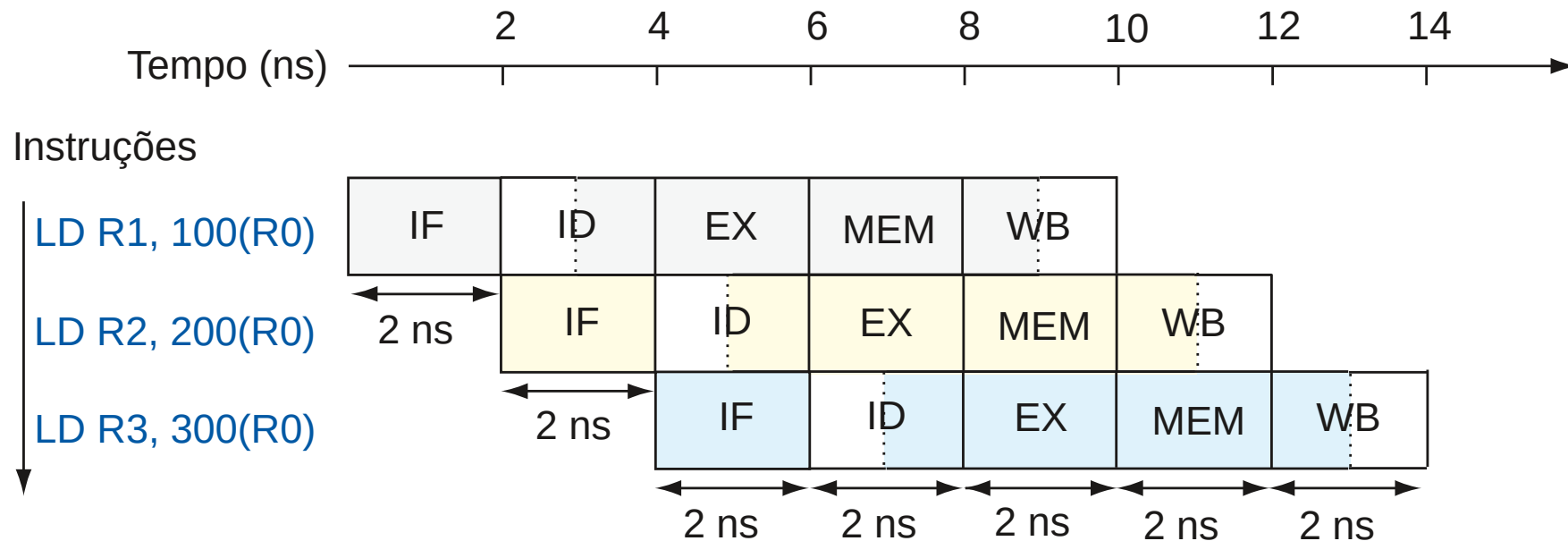
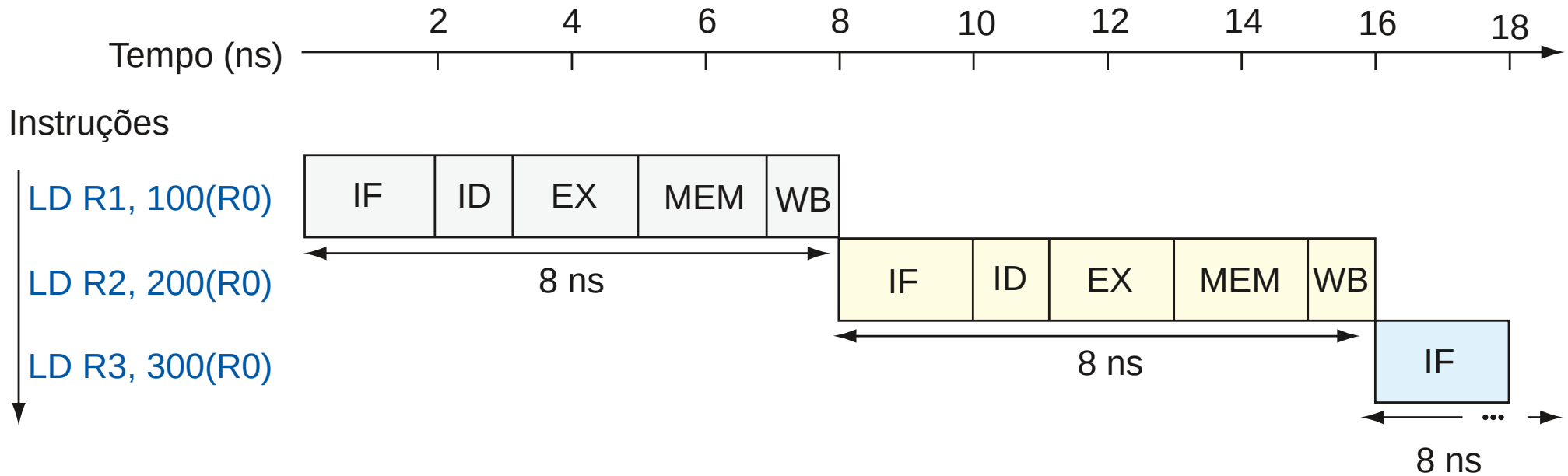


ILP e Pipeline

- **Instruction Level Parallelism (ILP):** Paralelismo entre as instruções
- **Técnica de pipeline:**
 - Técnica de implementação do processador (e de outras unidades funcionais)
 - **Sobrepõe a execução de múltiplas instruções:** Explora ILP
 - **Ideia:**
 - Execução da instrução é dividida em passos: **Estágios do pipeline**
 - Todos os estágios operam em paralelo, cada um com uma instrução diferente
 - Enquanto uma instrução executa em um estágio, outras instruções executam em outros estágios
 - **Vantagens ?**
 - **Desvantagem ?**

Exemplo 1: Execução de Instruções – Single-cycled × Pipeline



Desempenho do Pipeline

- **Tempo de execução de uma instrução:**
 - Não melhora no pipeline e pode piorar
- **Throughput** (nº de instruções terminadas por unidade de tempo):
 - Melhora no pipeline $\Rightarrow$ Pipeline melhora **tempo de execução do programa**
- **Condições ideais:**
 - Estágios do pipeline são balanceados
 - Há instruções suficientes para executar $\Rightarrow$ Transiente inicial desprezível
 - Não há conflitos entre instruções
 - Não há overhead de controle

Desempenho do Pipeline

- **Em condições ideais:**

$$speedup_{pipeline} = \frac{\text{tempo entre instruções}_{sem\ pipeline}}{\text{tempo entre instruções}_{pipeline}} = n^o \text{ estágios}$$

$$CPI_{pipeline} = 1$$

- **Em condições reais:**

$$speedup_{pipeline} \leq \frac{\text{tempo entre instruções}_{sem\ pipeline}}{\text{tempo entre instruções}_{pipeline}} < n^o \text{ estágios}$$

$$CPI_{pipeline} = 1 + \text{ciclos em stall por instrução}_{\text{causados por conflitos estruturais, de dados, de controle}} > 1$$

- **Por que não ter pipeline com MUITOS estágios ?**

Pipeline Básico

- Técnica simples para explorar ILP:
 - **Single issue** (despacho simples): inicia uma única instrução a cada ciclo
 - **In-order issue** (despacho em ordem): inicia instruções na ordem do programa
 - **In-order execution**: executa instruções na ordem do programa ⇒ **Escalonamento estático** (realizado pelo compilador)
 - **In-order completion**: termina instruções na ordem do programa
- **Exemplo 2:**

Instrução	Ciclos / Estágios								
	1	2	3	4	5	6	7	8	9
DADD ...	IF	ID	EX	Mem	WB				
DSUB ...		IF	ID	EX	Mem	WB			
DAND ...			IF	ID	EX	Mem	WB		
DOR ...				IF	ID	EX	Mem	WB	
DXOR ...					IF	ID	EX	Mem	WB

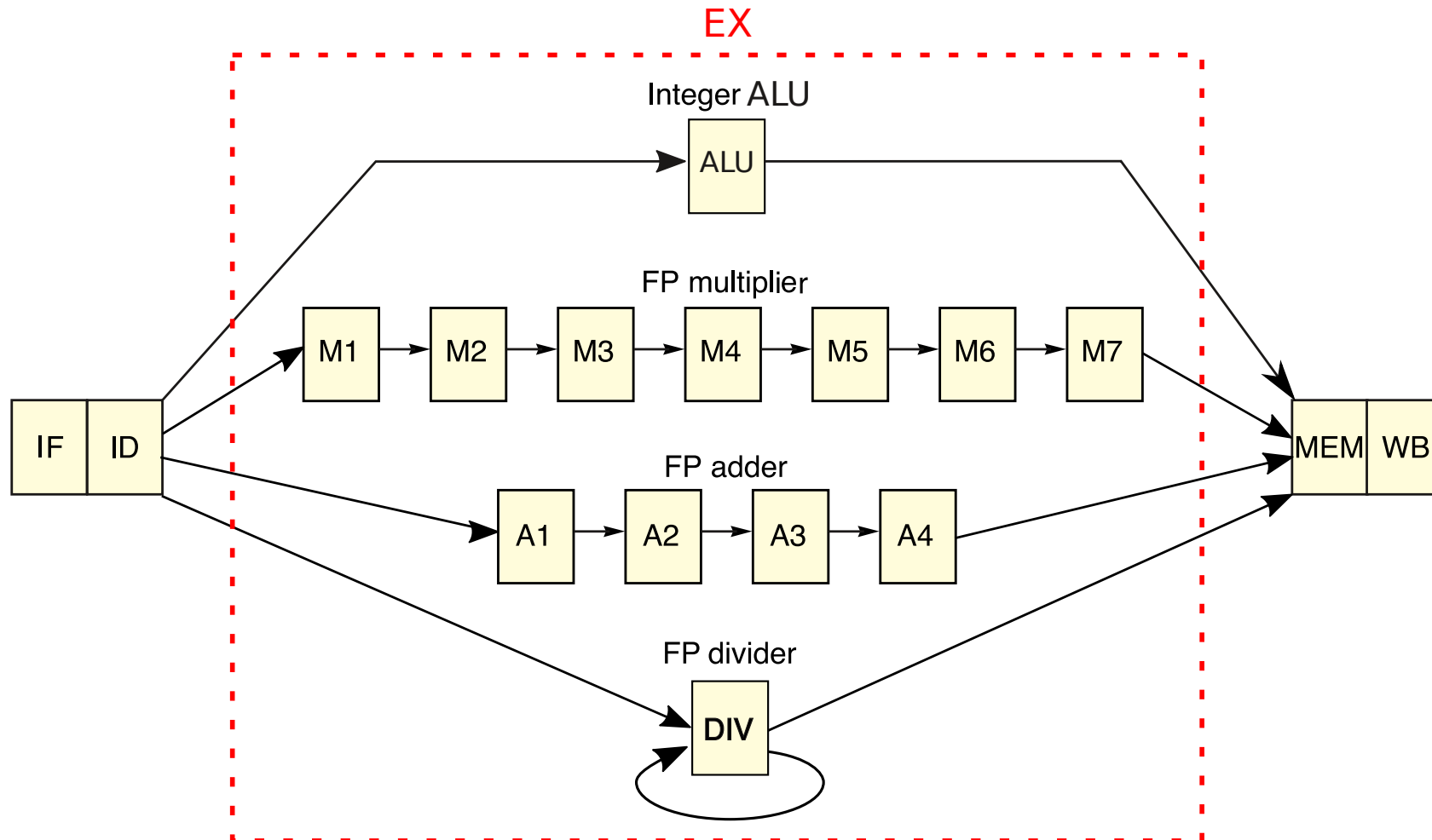
Pipeline com Operações Multiciclo

- Operações que consomem mais de 1 ciclo no estágio EX
- **Exemplos:**
 - Operações de multiplicação e divisão de inteiros
 - Operações aritméticas de ponto flutuante
- **Solução:**
 - Instrução ocupa estágio EX por vários ciclos para completar operação
 - Ter várias unidades funcionais (*pipelined* ou não)
- **Unidade Funcional (UF) que realiza uma operação possui:**
 - **Latência da operação:**
 - N^o de ciclos para produzir o resultado da operação
 - Afeta **conflito de dados**
 - **Intervalo de iniciação:**
 - N^o mínimo de ciclos entre a iniciação de 2 operações
 - Afeta **conflito estrutural**

Exemplo 3: Pipeline MIPS com Operações Multiciclo

4 unidades funcionais no estágio EX

Unidade funcional	Latência	Intervalo de iniciação	Observação
Integer ALU	1	1	—
FP multiplier	7	1	<i>pipelined</i>
FP adder	4	1	<i>pipelined</i>
FP divider	25	25	—



Exemplo 3: Execução de Operações Multiciclo no Pipeline

(a) Instruções independentes $\Rightarrow$ Não há conflitos de dados

Intervalo de iniciação do FP Adder = 1 ciclo $\Rightarrow$ Sem conflito estrutural

Instrução	Ciclos / Estágios											
	1	2	3	4	5	6	7	8	9	10	11	12
ADD.D F0, F2, F4	IF	ID	A ₁	A ₂	A ₃	A ₄	Mem	WB				
ADD.D F6, F2, F8		IF	ID	A ₁	A ₂	A ₃	A ₄	Mem	WB			

(b) **Conflito de dados** entre ADD.D e MUL.D:

- Latência do FP Adder = 4 ciclos $\Rightarrow$ **Stall**

Sem conflito estrutural

Instrução	Ciclos / Estágios														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ADD.D F0, F2, F4	IF	ID	A ₁	A ₂	A ₃	A ₄	Mem	WB							
MUL.D F6, F0, F8		IF	ID	stall	stall	stall	M ₁	M ₂	M ₃	M ₄	M ₅	M ₆	M ₇	Mem	WB

Exemplo 3: Execução de Operações Multiciclo no Pipeline

(c) Instruções independentes $\Rightarrow$ Não há conflitos de dados

Conflito estrutural no FP Divider:

- Intervalo de iniciação do FP Divider = 25 ciclos $\Rightarrow$ **Stall**

Instrução	Ciclos / Estágios													
	1	2	3	4	5	...	26	27	28	29	30	...	52	...
DIV.D F0, F2, F4	IF	ID	D ₁	D ₂	D ₃	...	D ₂₄	D ₂₅	Mem	WB				
DIV.D F6, F2, F8		IF	ID	stall	stall	...	stall	stall	D ₁	D ₂	D ₂	...	D ₂₅	...

(d) Conflito de dados entre MUL.D e ADD.D $\Rightarrow$ **Stall**

Sem conflito estrutural

Execução da SUB.D ?

Instrução	Ciclos / Estágios														
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
MUL.D F0, F2, F4	IF	ID	M ₁	M ₂	M ₃	M ₄	M ₅	M ₆	M ₇	Mem	WB				
ADD.D F6, F0, F8		IF	ID	stall	stall	stall	stall	stall	stall	A ₁	A ₂	A ₃	A ₄	Mem	WB
SUB.D F10, F2, F4			IF	?	?	?	?	?	?	?					

Instruction-Level Parallelism – ILP

- **Como melhorar mais o desempenho ?**
 - Explorar mais ILP
- **Como explorar mais ILP ?**
 - Encontrar no programa mais instruções que possam ser executadas simultaneamente
 - Mais instruções executando ao mesmo tempo $\Rightarrow$ Mais conflitos
 - Melhorar mecanismos para tratar conflitos

Dependências e Conflitos entre Instruções

- **Ordem do programa:**
 - Ordem das instruções determinada pelo programa
- **Dependência entre instruções:**
 - Propriedade do programa
 - Determina ordem em que instruções devem ser executadas para preservar semântica do programa
 - Pode causar conflito ou não
- **Conflito entre instruções** (hazard, perigo):
 - Propriedade da organização do processador
 - Há dependência entre as instruções e elas estão “perto” o suficiente $\Rightarrow$ Execução sobreposta ou reordenação das instruções afetaria o resultado
 - Pode causar atraso ou não
- **Atraso** (stall):
 - Propriedade da organização do processador
 - Limita ILP $\Rightarrow$ Reduz desempenho

Dependências e Conflitos entre Instruções

- Tipos de **dependências entre instruções**:
 - **Dependência de dados verdadeira**
 - **Dependência de dados falsa (dependência de nomes)**:
 - **Anti-dependência**
 - **Dependência de saída**
 - **Dependência de controle**
- Tipos de **conflitos entre instruções**:
 - **Conflito estrutural**
 - **Conflito de dados**:
 - **RAW** (Read After Write): Causado por dependência verdadeira
 - **WAR** (Write After Read): Causado por anti-dependência
 - **WAW** (Write After Write): Causado por dependência de saída
 - **Conflito de controle**:
 - Causado por dependência de controle

Dependência de Dados Verdadeira

- Instruções i e j , onde i precede j na ordem do programa:
 - j é dependente de dados de i se:
 i escreve em registrador ou posição de memória e j lê
Representado por: $i \rightarrow j$
- Há fluxo de dados entre as instruções:
 - Valor de um dado passa de i para j (por registrador ou memória)
- Exemplo 4: Dependência de dados verdadeira por registrador DADD $\rightarrow$ DSUB (R1)

```
DADD R1, R2, R3
DSUB R4, R1, R5
```
- Dependência verdadeira deve ser respeitada:
 - i deve executar antes de j
- Pode gerar conflito RAW (Read After Write)
- Soluções: Mantêm dependência, mas evitam conflito ou evitam/reduzem atraso
 - Forwarding
 - Escalonamento de código: estático e dinâmico

Dependência de Dados Verdadeira

- **Dependência de dados verdadeira:**

- **Por registrador:**

- Detecção simples: Comparação de campos c/ registradores origem/destino

- **Por posição de memória:**

- Detecção difícil: Endereço de memória não conhecido na compilação

- **Exemplo 5:**

```
SD  R1, 100 (R2)    # Mem[R2 + 100] = R1
```

```
...
```

```
LD  R3, 20 (R4)     # R3 = Mem[R4 + 20]
```

- Há dependência de dados verdadeira **por posição de memória** SD → LD ?
 - Se $R2 = 0$ e $R4 = 80$, há dependência

Dependência de Nomes (Dependência Falsa)

- 2 instruções usam mesmo nome (registrador ou posição de memória), mas **não** há fluxo de dados entre elas
- **Dependência falsa**: Não é dependência de dados verdadeira
- Instruções i e j , onde i precede j na ordem do programa:
 - **Anti-dependência**:
 - i lê registrador ou posição de memória e j escreve
 - **Pode gerar conflito WAR** (Write After Read)
 - **Dependência de saída**:
 - i e j escrevem no mesmo registrador ou posição de memória
 - **Pode gerar conflito WAW** (Write After Write)

- **Exemplos 6:**

Anti-dependência: DADD/DSUB (R2)

DADD R1, R2, R3

DSUB R2, R4, R3

Dependência de saída: DAND/DXOR (R6)

DAND R6, R7, R8

DOR R9, R6, R5

DXOR R6, R5, R7

- **Solução: Renomeação de registradores**

- Elimina dependência $\Rightarrow$ Instruções podem executar simultaneamente ou fora de ordem₁₅

Dependência de Controle

- Instruções i e j , onde i precede j na ordem do programa:
 - j é dependente de controle de i se:
 j deve ser executado ou não dependendo da execução de i
- Exemplo 7: DSUB é dependente de controle de BEQ

```
DADD    R1,    R2,    R3
```

```
BEQ     R4,    R0,    L
```

```
DSUB    R1,    R5,    R6
```

```
L:     . . .
```

- Pode gerar conflito de controle
- Soluções: Mantêm dependência, mas evitam conflito ou evitam/reduzem atraso
 - Instrução delayed branch
 - Predição de desvios: estática e dinâmica
 - Especulação por hardware
- Solução (elimina dependência de controle): Instruções condicionais

Exemplo 8: Dependências Verdadeiras

DADD	R1,	R2,	R10	% R1	:=	R2	+	R10
DSUB	R4,	R1,	R5	% R4	:=	R1	-	R5
DAND	R6,	R1,	R7	% R6	:=	R1	AND	R7
DOR	R8,	R1,	R9	% R8	:=	R1	OR	R9
LD	R3,	0	(R1)	% R3	:=	Mem[R1	+	0]
DXOR	R12,	R3,	R11	% R12	:=	R3	XOR	R11

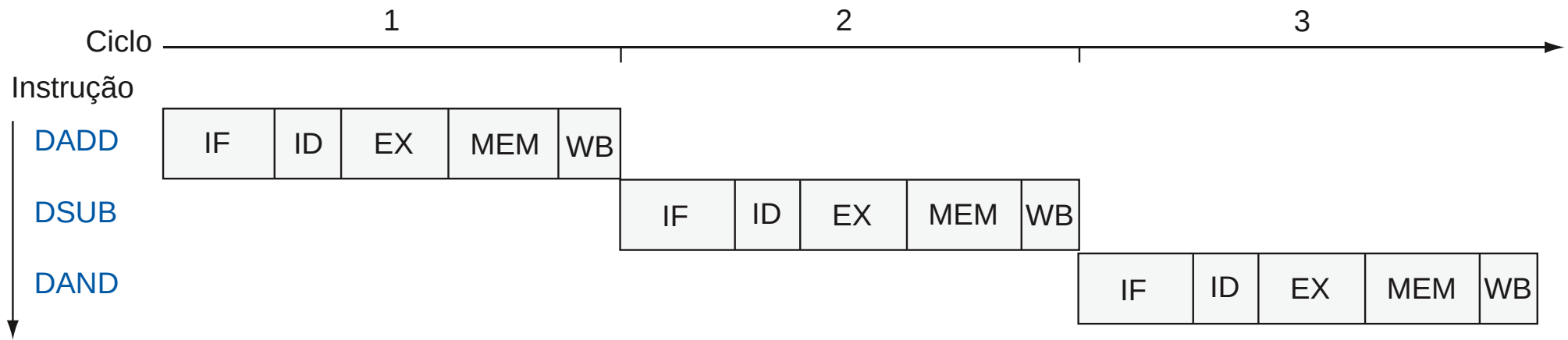
- **Dependências verdadeiras:**
 -
 -
 -
 -
 -
- **Não há anti-dependências**
- **Não há dependências de saída**

Exemplo 8: Execução no Processador MIPS Single-Cycled

DADD R1, R2, R10
DSUB R4, R1, R5
DAND R6, R1, R7
DOR R8, R1, R9
LD R3, 0 (R1)
DXOR R12, R3, R11

- **Dependências verdadeiras:**

- DADD → DSUB (R1)
- DADD → DAND (R1)
- DADD → DOR (R1)
- DADD → LD (R1)
- LD → DXOR (R3)



- **No processador MIPS single-cycled:**

- Dependências verdadeiras NÃO causam conflito RAW
- **Por quê ?**

Exemplo 8: Execução no Pipeline MIPS

```
DADD  R1,  R2,  R10
DSUB  R4,  R1,  R5
DAND  R6,  R1,  R7
DOR   R8,  R1,  R9
LD     R3,   0   (R1)
DXOR  R12, R3,  R11
```

- **Dependências verdadeiras:**

- DADD → DSUB (R1)
- DADD → DAND (R1)
- DADD → DOR (R1)
- DADD → LD (R1): **não** causa conflito RAW
Por quê ?

- LD → DXOR (R3): causa conflito RAW e **atraso**
Tratado com stall e forwarding

Instrução	Ciclos / Estágios										
	1	2	3	4	5	6	7	8	9	10	11
DADD ...	IF	ID	EX	Mem	WB						
DSUB ...		IF	ID	EX	Mem	WB					
DAND ...			IF	ID	EX	Mem	WB				
DOR ...				IF	ID	EX	Mem	WB			
LD ...					IF	ID	EX	Mem	WB		
DXOR ...						IF	ID	ID	EX	Mem	WB

- **No pipeline MIPS:** Dependências verdadeiras PODEM causar conflito RAW
 - **Por quê ?**

Exemplo 8: Execução no Pipeline MIPS

- **Com escalonamento de código:** Mantém dependência, porém evita atraso

- **Código escalonado:**

```

DADD  R1,  R2,  R10
DSUB  R4,  R1,  R5
DAND  R6,  R1,  R7
LD     R3,  0   (R1)
DOR   R8,  R1,  R9
DXOR  R12, R3,  R11
    
```

- **Dependências verdadeiras:**

- DADD → DSUB (R1): causa conflito RAW, usa forwarding, sem atraso
- DADD → DAND (R1): causa conflito RAW, usa forwarding, sem atraso
- DADD → DOR (R1): **não** causa conflito RAW
- DADD → LD (R1): causa conflito RAW, usa forwarding, sem atraso
- LD → DXOR (R3): causa conflito RAW, usa forwarding, sem atraso

Instrução	Ciclos / Estágios										
	1	2	3	4	5	6	7	8	9	10	11
DADD ...	IF	ID	EX	Mem	WB						
DSUB ...		IF	ID	EX	Mem	WB					
DAND ...			IF	ID	EX	Mem	WB				
LD ...				IF	ID	EX	Mem	WB			
DOR ...					IF	ID	EX	Mem	WB		
DXOR ...						IF	ID	EX	Mem	WB	

Exemplo 9: Dependências Falsas

DADD	R1,	R2,	R3	% R1 := R2 + R3
DSUB	R2,	R4,	R5	% R2 := R4 - R5
DAND	R6,	R7,	R8	% R6 := R7 AND R8
DOR	R9,	R6,	R10	% R9 := R6 OR R10
DXOR	R6,	R11,	R12	% R6 := R11 XOR R12

- **Dependência verdadeira:**

-

- **Anti-dependências:**

-

-

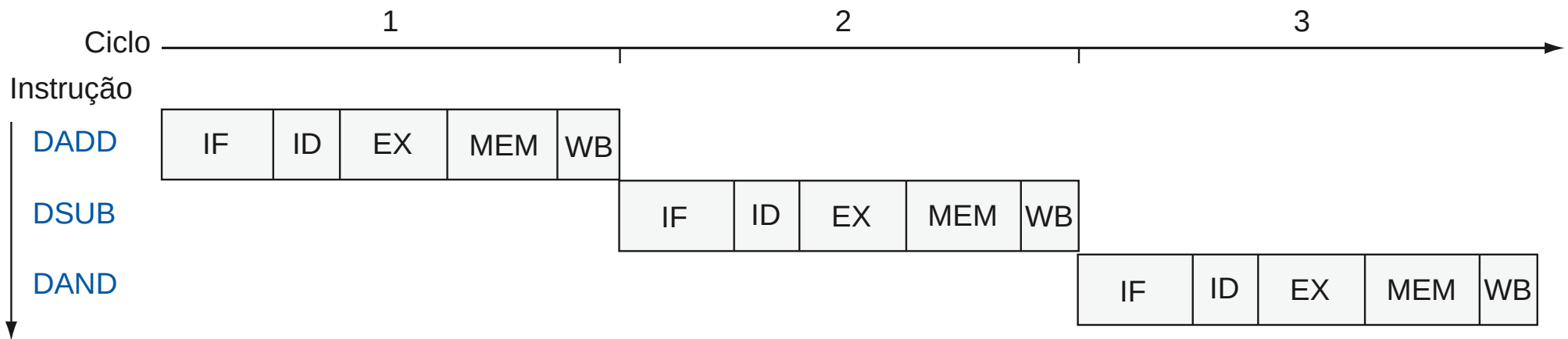
- **Dependência de saída:**

-

Exemplo 9: Execução no Processador MIPS Single-Cycled

DADD R1, R2, R3
DSUB R2, R4, R5
DAND R6, R7, R8
DOR R9, R6, R10
DXOR R6, R11, R12

- **Dependência verdadeira:**
 - DAND → DOR (R6)
- **Anti-dependências:**
 - DADD / DSUB (R2)
 - DOR / DXOR (R6)
- **Dependência de saída:**
 - DAND / DXOR (R6)



- **No processador MIPS single-cycled:**
 - Anti-dependências NÃO causam conflito WAR
 - Dependências de saída NÃO causam conflito WAW
 - **Por quê ?**

Exemplo 9: Execução no Pipeline MIPS

```
DADD  R1, R2, R3
DSUB  R2, R4, R5
DAND  R6, R7, R8
DOR   R9, R6, R10
DXOR  R6, R11, R12
```

- **Dependência verdadeira:**
 - DAND → DOR (R6)
- **Anti-dependências:**
 - DADD / DSUB (R2)
 - DOR / DXOR (R6)
- **Dependência de saída:**
 - DAND / DXOR (R6)

Instrução	Ciclos / Estágios								
	1	2	3	4	5	6	7	8	9
DADD ...	IF	ID	EX	Mem	WB				
DSUB ...		IF	ID	EX	Mem	WB			
DAND ...			IF	ID	EX	Mem	WB		
DOR ...				IF	ID	EX	Mem	WB	
DXOR ...					IF	ID	EX	Mem	WB

- **No pipeline MIPS:**
 - Anti-dependências NÃO causam conflito WAR
 - Dependências de saída NÃO causam conflito WAW
 - **Por quê ?**